

Received 30 October 2025, accepted 10 December 2025, date of publication 22 December 2025,
date of current version 29 December 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3647111

RESEARCH ARTICLE

GenScore: Agent-Based Short-Answer Question Generation and Scoring in Software Engineering Courses

TA NGUYEN BINH DUONG^{ID} AND LWIN KHIN SHAR^{ID}, (Member, IEEE)

School of Computing and Information Systems, Singapore Management University, Singapore 188065

Corresponding author: Ta Nguyen Binh Duong (donta@smu.edu.sg)

ABSTRACT Short-answer questions are commonly used in educational assessments, as they are often viewed as a more effective way than multiple-choice questions to determine whether students have achieved the intended learning outcomes. However, manually creating appropriate questions targeting different cognitive levels such as those defined by the Bloom's Taxonomy, and grading text answers from students are not trivial tasks for instructors. Existing work on auto-question generation and scoring in computing education typically targets coding-based questions. However, in software engineering courses, assessments can extend beyond coding to understanding of processes, DevOps methodologies, system design, etc. This work aims to address the dual problem of auto-generating and auto-scoring text based short-answer questions in software engineering courses. We propose an agent-based approach to question generation and scoring, in which each agent performs a distinct task and communicates with each other to complete its tasks. An agent can be realized using a selected large language model (LLM). To this end, we have implemented the proposed approach into a fully functional web app, utilizing OpenAI's GPT-3.5 and GPT-4o models, which was deployed to support instructors and students in our course. We conducted a thorough evaluation of the agent-based approach using several public and private datasets. The results demonstrated the effectiveness of our approach. Among other insights, we observed that the LLM agents are good at generating questions at Bloom's cognitive levels of *Remember*, *Understand*, *Apply*, and *Analyze*; and are good at scoring students' answers when provided with 3 or more actual examples of human scoring.

INDEX TERMS Agent based systems, large language models, short-answer questions, auto-scoring, question generation.

I. INTRODUCTION

Assessments in education can make use of multiple-choice questions, short-answer questions, coding tests, essay writing, etc. While options such as multiple-choice questions are great for large cohorts in which evaluation speed and consistency are needed, short-answer questions are useful when you need evidence that students can recall or analyze key steps or concepts covered in the course [1]. For such questions, students will usually need to provide answers in several lines of text in an open-ended format. In many

cases, short-answer questions are believed to be more effective than multiple-choice ones due to the former's greater requirement regarding full information retrieval and analysis when answering the questions [2], [3].

To be more effective in assessing learning, short-answer questions can be set at higher cognitive levels as specified by the Bloom's Taxonomy [4], which has been used widely in education. Questions that target *Analyze*, *Evaluate*, and *Create* (the top three tiers of the Bloom's Taxonomy) help learners to acquire a deeper understanding, long-term retention of knowledge, and a development of critical thinking and problem solving skills [5]. These questions provide a better differentiation between students who can

The associate editor coordinating the review of this manuscript and approving it for publication was Alba Amato^{ID}.

only recall facts and those who can actually grasp the content. In this way, instructors can foster learners who can remember key concepts and can use them effectively in new situations. For instance, in the software engineering course taught by the first author, in which foundational knowledge and theoretical aspects of software engineering are more important, students can be asked a short-answer question such as *briefly define continuous integration*, which is at the *Remember (Recall)* cognitive level of the Bloom's Taxonomy. On the other hand, a question such as *Given a team that frequently misses sprint goals, determine whether Scrum is suitable and briefly justify your recommendation* is clearly at a higher cognitive level (*Evaluate*) which requires more than just the ability to recall facts.

However, creating quality short-answer questions as described above and grading text answers from students manually can take a lot of time. Furthermore, students often want to practice on-demand, i.e., whenever they have the time, and require immediate feedback. For creating questions, instructors may wish to cater to students with varying levels of competency in a course, so the questions should be personalized at appropriate Bloom's Taxonomy levels. In addition, as such short-answer questions can have several possible answers which can be entirely or partially correct, manual scoring in this case is a tedious and time-consuming process [6]. This has been the challenge for software engineering courses at our institution where the number of computing students has been increasing steadily, which may also apply to many other institutions due to the increasing demand for IT professionals worldwide [7].

There has not been much research in auto-generating short-answer questions recently. In [8], the authors described the application of large language models (LLMs) and knowledge graphs to generate the questions and answers for cybersecurity training. In [9], questions were generated for content in Spanish textbooks and evaluated by subject matter experts. In another recent study [10], the authors used LLMs (GPT-4) to generate Bloom's Taxonomy-aligned multiple-choice questions (not short-answer) for a college-level biology course. We have not come across works on auto-generating short-answer questions for software engineering courses.

On the other hand, auto-scoring of short textual answers is a well-known problem in technology-enabled education. Many existing approaches made use of traditional machine learning techniques [11], [12], [13]. These approaches need to incorporate feature engineering and model training. Recent approaches used deep learning to reduce the effort of manual feature engineering, but deep learning requires a significant amount of quality data to work well. More recently, researchers have been using LLMs such as OpenAI's GPT family of models to perform auto-scoring of open-ended answers, e.g., [14], [15], [16], [17], and [18]. However, not much work has been done for LLM-based

auto-scoring of short answers in the context of computing education, especially in theory-oriented software engineering courses. Such courses cover a wide range of topics including processes, software design, automation and quality practices, etc. Short-answer questions in these topics could have many different correct or partially correct solutions. We note that auto-scoring methods in computing education typically focus on programming based questions [19], which could be evaluated by executing a suite of test cases.

This study attempts to address a real-world bottleneck in education in general: by automating both question generation and answer scoring, educators can deliver personalized assessments to a large number of students, and at the same time ensure that students receive immediate feedback needed for effective learning. Such automated systems have the potential to maintain objective grading standards. They can also enable educators to focus on other important activities such as curriculum design, personalized mentoring, etc. In this paper, we make the following major contributions:

- We consider the dual problem of auto-generating questions and auto-scoring short-answers in the context of more theory-oriented software engineering courses. We target courses that focus on software processes and DevOps methodology aspects of software engineering rather than programming topics, which have been studied more extensively from an assessment point of view.
- We propose an agent-based approach to question generation and scoring. Multiple agents are deployed in our system in which each performs a distinct function, communicates with each other to complete the generation or scoring task. In this work, an agent can be realized using a selected LLM, e.g., one of the OpenAI GPT models. Each agent is able to make use of suitable tools, e.g., file search, to complete its tasks.
- We design and implement a fully functional web based system, called GenScore, for our agent based approach. The system supports both instructors and students. Instructors can use the system to quickly generate questions and quizzes; while students can practice question answering on-demand with instant grading.
- We conduct an extensive evaluation of our approach using three datasets — a public dataset which has been widely used in short-answer scoring and two of our own datasets. We make use of a mixed-method strategy in this study which evaluates the performance of our approach both quantitatively and qualitatively. For the qualitative evaluation, a subject human expert assesses the quality of AI-generated outputs manually through many rounds of reviews. The results show the viability of our approach and provide further insights for future work in question generation and auto-scoring for various courses.

II. BACKGROUND

Generating and scoring open-ended, short-answer questions is a challenging problem as we need to ensure that they not only cover basic knowledge acquisition but also foster higher-order cognitive skills. As a metric to evaluate the generation-scoring efficacy, we leverage the widely-used Bloom's Taxonomy [4]. In education, the Bloom's Taxonomy organizes cognitive development into six hierarchical levels (or categories), namely *Remember*, *Understand*, *Apply*, *Analyze*, *Evaluate*, and *Create*. *Remember* is about recalling facts or terminology covered in the course; *Understand* requires students to explain or interpret known facts and terms; *Apply* asks students to put knowledge into usage in new contexts; *Analyze* requires students to break information into components and explore relationships between them, e.g., decomposing and analyzing a case study; *Evaluate* assesses critical judgment of students given some criteria; lastly, *Create* challenges students to synthesize and combine known elements to form new solutions. Table 1 summarizes the Bloom's categories and definitions used in this work.

TABLE 1. The Bloom's taxonomy.

Bloom's categories	Definition used
Remember	Recall facts and basic concepts
Understand	Explain ideas or concepts
Apply	Use information in new situations
Analyze	Draw connections among ideas
Evaluate	Justify a stand or decision
Create	Produce new or original work

The Bloom's Taxonomy has been applied to learning design and assessment in computer science education [20]. Here we show a few examples extracted from our course to illustrate how questions and answers under different Bloom levels can differ:

Remember. Ask students to list a value of the Agile Manifesto, for which the answer could simply be: *working software over comprehensive documentation*.

Understand. Ask students to explain what the above Agile value means, for which the answer could be more elaborated and take more effort, such as: *Documentation should not delay the early and frequent delivery of a usable product*.

Apply. Ask students to write user stories with proper acceptance criteria.

Analyze. Ask students to look at deployment log to identify process issues.

Evaluate. Ask students to criticize the choices of container orchestration tools.

Create. Ask students to propose a new way for measuring technical debts and its effects on team performance in a hypothetical organization.

III. PROPOSED APPROACH

Our approach aims to address the problem of generating valid, clear, and relevant short-answer questions; and to perform automatic scoring of the answers given by students

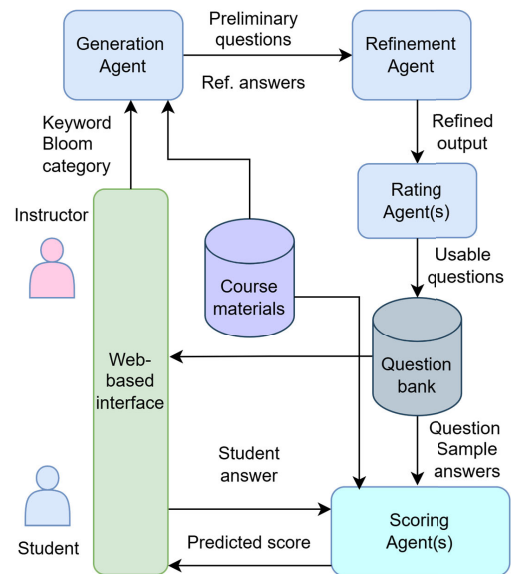


FIGURE 1. GenScore: short-answer question generation and scoring.

in our software engineering course, so that they can practice and get instant feedback. In the following, we describe our approach called GenScore.

A. DESIGN

The design of the GenScore system is shown in Figure 1. It has four types of agents, namely *Generation Agent*, *Refinement Agent*, *Rating Agent(s)*, and *Scoring Agent(s)*, each performing a distinct function and communicating with other agents in the system to complete the given task, i.e., question generation or scoring. An agent can be realized using a selected LLM [21]. In our implementation, we use OpenAI Assistants API, which enables each agent to use selected tools for its operations. For instance, our agents make use of file search for relevant context extraction from data sources such as lecture notes and other course materials, and JSON encoding so that each pair of generated question and answer can be extracted individually.

In GenScore, instructors use a web interface to generate questions, by submitting keywords and a Bloom's category as inputs. The *Generation Agent* automatically creates preliminary short-answer questions and reference answers based on the given keywords, Bloom's category, and selected course materials. The *Refinement Agent* then applies predefined quality criteria to improve each question. Finally, one or more *Rating Agents* evaluate the refined questions and reference answers using the same criteria used by the *Refinement Agent*. Questions passing a rating threshold, along with their model answers, are stored in the question bank for students to attempt later. By decomposing the question generation process into multiple stages, the system allows each agent in a stage to be more focused and effective on its tasks [22].

Students can practice the questions via a web interface in GenScore and submit short-answer responses. Each student

```

1 Generate a short answer question about <continuous
  integration> based on the content of the
  lecture notes <automation.pdf>. The question
  must belong to the <Remember> category of the
  Bloom taxonomy, i.e., it must test students on
  their ability to <recall fundamental facts or
  terminology>.
2 Provide two sample answers for the question.
3 Question and answers must be: <criteria>
4
5 RESPONSE FORMAT:
6 Question:...
7 Answer(s)
8 1)...
9 2)...
```

LISTING 1. Key parts of the generation agent's prompt.

```

1 1) Valid: correct and belong to the <Remember>
  category of the Bloom taxonomy.
2 2) Clear: questions and reference answers must be
  clear and concise.
3 3) Relevant: within the scope of the attached
  lecture notes <automation.pdf>.
```

LISTING 2. Criteria for question generation.

answer is graded by a predefined number of *Scoring Agents*, which could be more reliable than a single agent. Similar to the above, the Scoring Agents also make use of tools such as file search to obtain relevant context from given lecture notes for scoring a given question. The student receives the predicted score immediately via the web interface. In the following, we discuss in detail of how our agents work.

B. GENERATION AGENT

The Generation Agent can produce a user-specified number of questions that focus on a given keyword and a chosen Bloom's taxonomy category. For a keyword it retrieves the relevant context from lecture notes, which are stored as embedding vectors. It then generates the question and a predefined number of reference answers. If no keywords are supplied, it will generate questions on any concept found in the lecture notes. Listing 1 shows the LLM prompt used by the Generation Agent — line 3 provides the instruction regarding the criteria that the generated question and its answers should satisfy. Listing 2 provides details on the criteria used in this work, namely validity, clarity, and relevance. The same criteria are also used in the Refinement Agent and Rating Agent(s). The Generation Agent prompt also includes other rules for obtaining a specific output format.

C. REFINEMENT AGENT

Motivated by increasing use of refinement agents in various applications [23], [24], we use a Refinement Agent to allow the Generation Agent to reflect on its original output and possibly make appropriate changes. Each output generated by the Generation Agent will go through a predetermined number of rounds of improvement by the Refinement Agent. Our aim is to enhance the generated questions and reference answers according to the given quality criteria while minimizing human intervention. In this way, the Generation Agent

```

1 A short answer question about <continuous
  integration> is given below. It is supposed to
  belong to the <Remember> category of the
  Bloom taxonomy, i.e., it must be testing
  students on their ability to <recall
  fundamental facts or terminology>. Improve it
  using the content of the lecture notes <
  automation.pdf>.
2
3 The question has sample answers under Answer(s),
  which should be improved as well. The overall
  concepts tested in the given question should
  not be changed. The following criteria should
  be considered for improvement: <criteria>
4
5 If there is nothing to improve, ONLY reply with "
  Nothing to improve".
6
7 <Question and answers to be refined>
8
9 <Response format>
```

LISTING 3. Key parts of the refinement agent's prompt.

```

1 A short answer question about <topic> is given
  here. Question: <question text>. Answer: <
  reference answer text>.
2 This question and its answer are supposed to
  belong to the <Remember> category of the Bloom
  taxonomy, i.e., it must be testing students
  on their ability to <recall facts and basic
  concepts>.
3 Rate the quality of the question and its answers
  using following criteria: <criteria>.
4
5 You can award a maximum of 2 points for each of
  the criteria above, for a total of 12 points.
  Points are awarded based on how well the
  question and its answer meet each of the
  criteria:
6 1) Not at all: 0 point
7 2) Partially satisfactory: 1 point
8 3) Entirely satisfactory: 2 points
```

LISTING 4. Key parts of the rating agent's prompt.

increases the likelihood that students can immediately use the generated questions for self-practice without requiring additional manual review by an instructor. The Refinement Agent is given access to the same embedding vector stores, and is instructed to return the improved output in the same format. If it concludes that no further improvement can be made with respect to the criteria, it returns *Nothing to improve*. Listing 3 shows the prompt used by the Refinement Agent.

D. RATING AGENTS

A question - reference answer pair is then sent to a predefined number of Rating Agents which independently evaluate and compute a numeric rating of each generated question and its corresponding reference answers. To ensure the quality of questions, a newly generated question is considered usable if and only if all the Rating Agents award the full rating, otherwise it will be discarded. To be given the full rating, a question and its reference answers must be valid, clear, and relevant as defined in Listing 2. Each criterion can be given a maximum of two points as shown in Listing 4.

```

1 # Provide few-shot examples, more can be given
  using additional messages
2 Message 1: {"role": "user", "content": "Given the
  question <question text>, provide a score out
  of <max score> for the corresponding answer <
  answer text> in the JSON format {"score": (
  score)}."}
3 Message 2: {"role": "assistant", "content": "{"
  score": <score given by human>"}"
4
5 # This message is used for auto-grading
6 Last message: {"role": "user", "content": "Given
  the question <question text>, provide a score
  out of <max score> for the corresponding
  answer <answer text> in the JSON format {"
  score": (score)}."}
7
8 # The LLM will respond with a predicted score
9 Message: {"role": "assistant", "content": "{"score
  ": <predicted score>"}"}

```

LISTING 5. Key parts of the scoring agent's prompt.

E. SCORING AGENTS

For a given pair of question and student answer to be scored, the Scoring Agent uses a few-shot prompt [25] with examples extracted from the human-graded answers, as shown in Listing 5. For better representation of examples, they are selected from three different score categories, i.e., low score, medium score, and high score [14]. The number of examples in each run can be varied, e.g., 3, 6, or 9 examples can be selected. The Scoring Agent can be instructed to make use of suitable tools, e.g., file search, to incorporate relevant context from the lecture notes into the prompt if needed. As using a single Scoring Agent may produce inconsistencies in predicted scores due to the stochastic nature of LLMs, we propose the use of multiple independent Scoring Agents in scoring the same answer. The final score is the average of all the individual scores produced by each agent.

IV. IMPLEMENTATION

The GenScore system comprises two microservices exposing RESTful APIs for auto-generation and auto-scoring, respectively. The web interface was implemented with NextJS. Both microservices use the OpenAI Assistants API with either GPT-3.5-turbo or GPT-4o-mini for vector stores, message threads and runs. The services share a MongoDB instance for storing generated questions and scores.

Users can specify their role as “Instructor” or “Student” on the landing page of the web app. The instructor can create a new topic for the course by uploading relevant lecture notes and sample questions files in PDF and Excel file format respectively. They can also view and edit the existing questions (Fig. 2), or generate new ones based on the lecture notes and sample questions available (Fig. 3). These files can be updated or downloaded through the interface. Questions are generated individually by providing a keyword and a desired Bloom's category. The generated questions can be exported into an Excel file suitable for importing

Short Answer Questions

FIGURE 2. Instructors can view, add, edit, and delete existing questions.

FIGURE 3. Question generation interface.

FIGURE 4. Score and feedback generated by the auto-scoring service.

into Wooclap (<https://www.wooclap.com>), an interactive quiz platform widely used by educators [26].

Students can view and search for course topics and relevant questions, then start a quiz to practice. They can enter their answers to the respective question and evaluate their answers immediately with the auto-scoring service. Feedback will be provided together with a numerical score and sample reference answers, as seen in Fig. 4. In this work, we focus on evaluating the quality of the generated questions, and the accuracy of the auto-scoring approaches, but not the quality of the feedback.

More details on how GenScore system works can be found in the video demo included with the submission of this paper.

TABLE 2. Lecture notes incorporated as grading contexts.

Topic	Keywords	Pages
Automation	version control, CICD, feedback loops, production environments, git workflow, branching, merging, infrastructure-as-code, updating strategies, blue-green deployment, canary deployment, feature flags	25
Software design	dependency injection, circular dependency, monolithic, microservices, REST, HTTP	32
Software processes	waterfall, Scrum, sprint, user story, story point estimation	29
Security	CIA triad, encryption and hashing, access control, immutable infrastructure, backup and recovery, input validation, secure design principles	30
Versioning	history graph, commit, head, object graph	36
XP practices	pair programming, code review, refactoring, code smells, technical debts, pairing styles	30
Software support	ITIL framework, incidents, problems	50
Software testing	test driven development, blackbox, white-box, input space partitioning, boundary value analysis, code coverage, unit testing, mocking, integration testing, acceptance test	22

V. EVALUATION METHODOLOGY

We aim to answer the following research questions:

- **RQ1:** How effective are the question generation approaches?
- **RQ2:** How do ratings of generated questions by LLM based agents compare to human ratings?
- **RQ3:** How does the single agent approach compare to the existing auto-scoring approaches?
- **RQ4:** How does the multi-agent scoring approach compare to the single agent approach?

A. DATASETS

1) LECTURE NOTES FOR CONTEXT

We used PDF lecture notes in our software engineering course to provide relevant context for both question generation and answer scoring. In addition, for question generation, we used a list of keywords for each lecture topic to mitigate duplicated questions across runs. Each keyword was used to generate a set of six questions corresponding to six different levels in Bloom's Taxonomy. The lecture topics and list of keywords used for each topic are shown in Table 2.

Three datasets were used to evaluate the performance of our proposed auto-scoring approaches.

2) MOHLER DATASET [11]

This dataset has been widely used in evaluating short-answer auto-scoring approaches. We use this dataset for a fair comparison to existing methods [14]. We note that the questions in this dataset are not strictly about software engineering content, but they cover software design and some programming concepts. The dataset was obtained through exams/assignments given to students in an introductory computer science class at the University of North Texas. Each

TABLE 3. Mohler dataset: Sample Question, answers and scores.

Question	What stages in the software life cycle are influenced by the testing stage?
Reference Answer	The testing stage can influence both the coding stage (phase 5) and the solution refinement stage (phase 7).
Student Answer	Verification, coding, refining the solution and maintenance are all influenced by the testing stage.
Score 1	4
Score 2	5
Average Score	4.5

TABLE 4. SE dataset: Sample question, answers and score.

Subtopic	Automation
Question	What is one advantage of canary deployment?
Reference Answer	Can minimize the impact of errors to a subset of users
Graded Answer	it is cheaper to do
Graded Answer's Score	1

student answer was marked by two graders and the average mark (out of a maximum 5) was calculated for each answer. The dataset has 87 questions, each with a reference answer. We excluded 6 questions from the experiments as they are not short-answer questions. There are 24 to 31 student answers per question, totaling 2273 answers. Table 3 shows a sample question/answers/scores in this dataset. For auto-scoring, we split the answers in this dataset into 3 different categories according to their marks: low (less than or equal to 2 marks), medium (less than or equal to 4 marks) and high (5 marks). In this way, we could use examples under different categories to improve LLM based scoring.

3) SOFTWARE ENGINEERING (SE) DATASET

This dataset has been used in [14]. It contains short answers to questions on SE topics listed in Table 2, which are covered in our course. We could make use of the available PDF lecture notes to provide relevant context in auto-scoring the answers in this dataset. It is complementary to the Mohler dataset which is more about programming concepts. In summary, this dataset has 32 short-answer questions. Each question has 1-4 reference answers, and an average of 13 graded answers. In total, there are 421 graded answers with marks ranging from 0 to 4. We split the answers in this dataset into 3 mark categories: low (less than or equal to 1 marks), medium (less than or equal to 3.5 marks) and high (4 marks). Table 4 shows an example extracted from this dataset. This dataset mostly consists of questions under the lower cognitive levels of the Bloom's Taxonomy, namely *Remember* and *Understand* (30 out of 31 questions in this dataset are classified under these two levels).

4) GENERATED DATASET

We aimed to study the effect of different Bloom's levels on the auto-scoring performance. For this purpose, we selected 6 questions out of all the questions generated by the

TABLE 5. Generated dataset: Sample question, answers and score.

Subtopic	Automation
Question	How can the concepts of version control and branching strategies facilitate effective merging of changes in a collaborative software development environment?
Bloom's Level	Analyze
Reference Answer	Branching strategies, such as feature branches, parallelize the development process by allowing teams to experiment independently without affecting the main codebase. This means each developer can resolve their changes locally, and once ready, they can initiate a merge into the main branch. The version control system then provides tools to reconcile differences and automatically merge content wherever possible, simplifying the overall integration process and reducing the risk of overwriting changes inadvertently.
Graded Answer	Branching helps in organizing work. This makes merging easy.
Graded Answer's Score	2

Generation Agent as described in Section III. Out of the 6 questions, 3 are under the topic of Automation, and the other 3 are under the topic of Security. The questions belong to three different Bloom's cognitive levels, namely *Remember* (2 questions), *Analyze* (2 questions), and *Create* (2 questions). The selected questions and their reference answers were evaluated by a human rater to be usable. Each question has a maximum mark of 4. We then simulated student answers at various mark categories (low, medium, and high, as described earlier) to each of these questions. In total, there are 22 answers to each question. A sample question and its corresponding answers/scores extracted from this dataset are shown in Table 5.

Similar to the SE dataset, each topic in the Generated dataset has a corresponding PDF deck which was used to provide additional context during auto-scoring, as shown in Table 2.

B. PERFORMANCE MEASURES

1) EVALUATING QUESTION GENERATION PERFORMANCE

For a consistent evaluation of the generated questions and reference answers, a single human rater (the 1st author of this paper) with significant experience in teaching software engineering evaluated all of them and provided a score based on the same criteria used by the Rating Agents:

- 1) **Validity of question:** whether the question is set correctly and belongs to the requested category of the Bloom's Taxonomy.
- 2) **Validity of answers:** whether all of the reference answers are correct and belong to the requested category of the Bloom's Taxonomy.
- 3) **Clarity of question:** whether the question is clear and concise.
- 4) **Clarity of answers:** whether all the reference answers are clear and concise.

- 5) **Relevance of question:** whether the question is within the scope of the specified lecture notes.
- 6) **Relevance of answers:** whether all the answers are within the scope of the specified lecture notes.

The human rater awarded a maximum of two points for each of the 6 criteria above, for a total of 12 points. Points were awarded depending on how well the question and its reference answers meet each criterion:

- 1) **Not at all: 0 point.** For example, a question designated for the Create category, but asks student to recall a concept will be given 0 point for Validity.
- 2) **Partially satisfactory: 1 point.** The question or answers might have some minor issues which require further modification before use. For example, some reference answers generated in our experiments, especially those under the Create category, provided additional details not covered in the lecture notes at all.
- 3) **Entirely satisfactory: 2 points.** The question or answers are without any issues and can be used right away with no further modification.

To mitigate the issue of not having a second rater, the following step-by-step evaluation process was followed to ensure the consistency of the ratings produced by the single human rater.

- 1) The rater focuses on a single course topic at a time, such as software testing, to facilitate more accurate Bloom category assignment through comparison of questions within the same topic.
- 2) The rater begins with questions in the simplest unrated Bloom category, Remember, which is one of the easiest cognitive levels to identify. The rater reads each question and evaluates its reference answers to assess validity, including correctness and appropriate Bloom category assignment. Points (0, 1, or 2) are then given accordingly.
- 3) The rater then evaluates the clarity (clear, conciseness) of the question and answers. Points are given accordingly.
- 4) The rater then refers to the relevant lecture notes to determine whether the question falls within the course scope and assigns points accordingly.
- 5) After completing ratings for all questions within the same topic and Bloom category, the rater compares these questions and their ratings both against each other and against questions from the immediately preceding Bloom category to identify potential discrepancies. For example, an Apply question must assess students at a higher cognitive level than Understand by testing how they use information in new situations rather than just explaining concepts. The rater then adjusts the ratings if needed.
- 6) The rater moves to the next Bloom category, e.g., Understand, in the same course topic and repeats steps 2-5.

- 7) After completing ratings for all Bloom categories within the same course topic, the rater compares questions across different categories to verify correct category assignment. The rater then adjusts ratings if needed.

Following the above process, the single human rater completed rating of 312 generated questions and reference answers in about 2 weeks.

2) EVALUATING SCORING PERFORMANCE

Similar to existing work in this area [27], the auto-scoring performance was evaluated using the Pearson correlation coefficient, mean absolute error (MAE) and root mean square error (RMSE).

The **Pearson correlation coefficient**, i.e., Pearson's r , is a common way to measure linear correlations with values ranging from -1 to 1. The values depend on the strength and direction of the relationship between two variables, i.e., a larger absolute value means stronger correlation.

$$r = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2 \sum_{i=1}^n (y_i - \bar{y})^2}} \quad (1)$$

where x_i represents the actual mark given by human graders, and y_i represent the mark given by the auto-scoring method for the same answer. $\bar{x}$ and $\bar{y}$ represent the means of x and y , respectively.

The **mean absolute error** (MAE) is the average of the absolute differences between the actual and predicted marks:

$$MAE = \frac{1}{n} \sum_{i=1}^n |x_i - y_i| \quad (2)$$

The **root mean square error** (RMSE) is also a popular measure for the quality of predictions:

$$RMSE = \sqrt{\frac{\sum_{i=1}^n (x_i - y_i)^2}{n}} \quad (3)$$

In Equations (2) and (3), n denotes the total number of answers under consideration, x_i is the true score for the i -th response, and y_i is the score predicted for that response by the auto-scoring system. RMSE and MAE are both widely adopted metrics for evaluating prediction accuracy in machine learning.

When using multi-agent scoring, we would need to know the level of agreement between different LLM based scorers to ensure the robustness of the scoring approach. For this, we computed the **inter-rater reliability** (IRR) metrics for the scorers involved using **percentage agreement** [28] and **Krippendorff's Alpha** [29]. Percentage agreement is a simplistic measure of IRR, which computes the percentage of occasions scorers agree with each other on the mark given to an answer out of the total number of given scores. Percentage agreement has the advantage of being easy to compute and interpret. However, it does not account for agreement occurring by chance. On the other hand, Krippendorff's

Alpha is a more robust IRR measure by adjusting for the agreement expected by chance, and can handle missing data, e.g., in some cases one of the LLM based scorers crashed and could not provide a score. Krippendorff's Alpha values range from -1 to 1. A negative value shows systematic disagreement, while a value closer to 1 indicates strong agreement.

C. LLM MODELS AND PARAMETERS

We used two different OpenAI LLMs in the experiments, namely GPT-3.5-turbo and GPT-4o-mini. These two LLMs were selected due to their relatively low costs compared to newer models. In particular, GPT-3.5-turbo costs \$0.5 and \$1.5 for 1 million input and output tokens, respectively. On the other hand, GPT-4o-mini costs \$0.15 and \$0.6 for the same numbers of input and output tokens, respectively. We noted that GPT-4o-mini, being a newer but smaller model, costs much less than GPT-3.5-turbo, which was selected to ensure a fair comparison to existing auto-scoring approaches [14]. For all models, we used the default setting in the OpenAI Assistants API for parameters such as temperature (1.0), max tokens, etc.

For experiments on question generation, refinement and rating, only GPT-4o-mini was used as it is a newer and cheaper model. For auto-scoring, we used both GPT-4o-mini (for better performance and cost), as well as GPT-3.5-turbo (for a direct comparison to existing LLM-based auto-scoring approaches).

VI. RESULTS AND ANALYSIS

A. QUESTION GENERATION

1) RQ1: PERFORMANCE OF THE GENERATION APPROACHES

We first analyzed the performance of our proposed generation approach when only the Generation Agent is used, i.e., without any further refinement on the initial output. For each keyword in Table 2, we generated six different questions for each Bloom's categories: *Remember*, *Understand*, *Apply*, *Analyze*, *Evaluate*, and *Create* [4]. A total of 312 pairs of question - reference answers were generated across the course topics. A sample of a question and its corresponding reference answer generated by the Generation Agent are shown in Listing 6.

Table 6 shows that the generated questions and their corresponding answers were mostly highly rated by the human rater across most of the selected software engineering topics. That is, the total scores shown in the second column were quite close to the full rating of 12 points. On the other hand, we note that questions in some topics such as Automation and Process had lower ratings compared to others.

We then looked at the individual components for a better understanding of the results. As observed from columns 3 to 5 of Table 6, there are no major issues with the generated questions in terms of validity, conciseness, and

```

1 Requested Bloom's category: Apply
2
3 Question: Imagine you have made some changes to a
  file in your Git repository, but now you want
  to revert back to the previous version of that
  file. Describe the steps you would take to
  accomplish this using Git commands.
4
5 Reference Answer: To revert back to the previous
  version of a file using Git, first, ensure you
  are in the repository where the file is
  located. You can check the current state of
  the files with 'git status'. Next, to restore
  the previously committed version of the file,
  you will use the command 'git checkout <file
  >', replacing '<file>' with the name of the
  file you wish to revert. This command will
  discard all changes made to the file in the
  working directory and reset it to the last
  committed state. Finally, you can confirm the
  reversion by using 'git status' again to
  ensure that no changes are indicated for that
  file.

```

LISTING 6. Sample question/reference answer, topic of Versioning.

TABLE 6. Rating of generated questions and reference answers. The second column show the average rating out of 12 points given by the human rater for all pairs of generated question-reference answers in each topic. The scores given to each question (columns 3-5) and its reference answers (columns 6-8) cover the validity (val.), clarity (cla.), and relevance (rel.) criteria as defined in Listing 2, where each criterion is given a maximum of 2 points.

Topic	total	questions			reference answers		
		val.	cla.	rel.	val.	cla.	rel.
Overall	11.49	1.79	1.99	1.98	1.74	1.98	1.98
Automation	11.12	1.64	2.0	1.97	1.54	2.0	1.97
Design	11.86	1.94	2.0	2.0	1.91	2.0	2.0
Support	11.78	1.89	2.0	2.0	1.89	2.0	2.0
Security	11.62	1.88	2.0	1.97	1.78	2.0	1.97
Process	11.23	1.6	2.0	2.0	1.63	2.0	2.0
XP	11.78	1.89	2.0	2.0	1.89	2.0	2.0
Testing	11.41	1.78	1.98	2.0	1.7	1.94	2.0
Versioning	11.67	1.92	2.0	1.96	1.87	1.96	1.96

relevance for many of the topics. However, the questions generated for topics like Automation, Process and Testing were rated lower for the validity criterion as a number of them did not belong to the requested Bloom's categories, especially the higher cognitive levels of *Evaluate* and *Create*. Similarly, we note that the relevance and clarity of most of the generated reference answers were generally good. On the other hand, some of the generated answers for *Evaluate* and *Create* levels were not considered valid as they were more appropriately classified under lower cognitive levels such as *Apply*. A typical example of such issue is discussed below.

Table 7 shows the ratings given by the human rater for the generated questions and answers, organized by their Bloom's Taxonomy levels. We observed that the ratings were consistently good across Bloom levels, except for *Evaluate* and *Create*. This is because, on topics such as Automation, Process and Testing where a large number of generated questions are mostly about explaining or applying well-known software engineering practices, it is more challenging to create valid questions in the higher cognitive levels of the Bloom's Taxonomy. As a result, the questions generated by

```

1 Question: Design a CI/CD pipeline for a
  hypothetical software application, explaining
  each stage and its purpose in the integration
  and delivery process.
2
3 Reference Answer: The CI/CD pipeline for the
  hypothetical software application starts with
  the code being pushed to a version control
  system such as Git. Upon a new commit, a
  Continuous Integration (CI) server is
  triggered, initiating the build process. This
  stage compiles the code and generates
  executable artifacts. Following the build,
  automated testing is conducted, which includes
  unit tests, integration tests, and security
  scans to ensure the quality and security of
  the code. If all tests pass, the application
  is deployed to a staging environment that
  closely mirrors the production environment. In
  the final stage, after user acceptance
  testing in the staging environment, the code
  is manually or automatically deployed to
  production, making it live for users.

```

LISTING 7. Sample question and reference answer, topic of automation.

TABLE 7. Average ratings given by the human rater for the generated questions and answers in their corresponding Bloom's categories.

Bloom's categories	total	questions			answers		
		val.	cla.	rel.	val.	cla.	rel.
Remember	11.65	1.82	2.0	2.0	1.82	2.0	2.0
Understand	11.88	1.94	2.0	1.98	1.96	2.0	2.0
Apply	11.9	1.98	2.0	1.98	1.98	2.0	1.96
Analyze	11.73	1.9	2.0	2.0	1.84	1.98	2.0
Evaluate	11.42	1.79	1.98	1.98	1.73	1.96	1.98
Create	10.34	1.31	2.0	1.98	1.1	1.98	1.98

the Generation Agent for these categories are more about *Remember*, *Understand*, and *Apply*. For example, when we requested a *Create*-level question in the topic of Automation with the keyword "CI/CD", the Generation Agent responded with the question and reference answer shown in Listing 7:

We could see that the question in Listing 7 is simply about describing a standard process (which is lower on Bloom's Taxonomy cognitive levels). Similarly, the reference answer describes a typical, generic CI/CD pipeline. It does not invent, customize, or adapt a pipeline for a specific, novel scenario; rather it just recalls and explains established best practices. In this case, we note that the question and its reference answer demonstrate *Remember* and/or *Understand* cognitive levels (and somewhat *Apply*), but they are not at the *Create* level as requested in the generation prompt.

We then evaluated the usage of the Refinement Agent to see whether it could actually improve the quality of the generated questions. We selected 72 pairs of question-reference answers that had been generated earlier on the topic of Automation, and refined them using the Refinement Agent. Note that the Refinement Agent aims to improve the quality of generated questions and answers by adjusting them according to the criteria as defined in Listing 2. Listing 8 shows a sample of the original and refined question using three rounds of refinement for an *Evaluate*-level question in the topic of Automation.

While the two questions in Listing 8 remain semantically equivalent, we could see that some phrases, such as "... in the

```
1 OUTPUT OF THE GENERATION AGENT:
2
3 Initial question: Based on the principles of CI/CD
  discussed in the lecture notes, justify the
  importance of having a well-structured
  deployment pipeline in software development.
4
5 Initial reference answer: A well-structured
  deployment pipeline is essential in software
  development as it allows for the automation of
  build, test, and deployment processes,
  ensuring that code changes can be delivered to
  production safely and rapidly. This structure
  minimizes the risk of errors associated with
  manual deployments and provides a consistent
  approach to managing software releases.
  Additionally, having a deployment pipeline
  facilitates continuous feedback and
  improvement, allowing developers to make
  quicker adjustments based on real user
  interactions and experiences, ultimately
  enhancing the quality and reliability of
  software releases.
6
7 OUTPUT OF THE REFINEMENT AGENT:
8
9 Refined question: Evaluate the importance of
  maintaining a structured deployment pipeline
  in accordance with CI/CD principles in
  software development.
10
11 Refined reference answer: A structured deployment
  pipeline is essential in software development
  because it automates the critical phases of
  building, testing, and deploying code. This
  automation not only improves the speed and
  efficiency of releasing code to production,
  but it also significantly reduces the risks
  associated with manual processes. Establishing
  a standardized pipeline fosters continuous
  feedback, allowing developers to swiftly
  incorporate user input, which is vital for
  enhancing the overall quality and stability of
  the software.
```

LISTING 8. Initial and refined question.

lecture notes”, which are usually redundant in an assessment setting, are omitted in the refined question. As a result, the refined question and its reference answer become more concise compared to the initial one. This kind of minor improvement has been observed in some instances where the original output was not fully satisfactory. However, such cases accounted for a small subset as the initial question and answers were already clear and concise in most cases.

Table 8 shows the scores obtained from the human rating process across the quality criteria — validity, clarity, relevance (Listing 2) — for the initial questions & answers vs refined ones. The first data row in the table shows the overall average rating (out of a maximum of 12 points) given to all questions and their corresponding reference answers. The next rows show the individual rating for each criterion evaluated during the human rating process. We could only observe marginal improvement by the Refinement Agent for most of the generated questions. The main issue was on question and answer validity, i.e., they did not belong to the correct Bloom’s categories such as *Evaluate* and *Create*. However, in such situations the Refinement Agents could not

TABLE 8. Respective scores for the initial and refined outputs in question generation. The initial output was already highly scored in many cases, therefore we only observed marginal improvement in the refined output. The topic is Automation. The scores were computed for 72 generated question-answer pairs.

Criteria	Initial	Refined
Overall	11.12	11.26
question’s validity	1.64	1.69
question’s clarity	2.0	2.0
question’s relevance	1.97	1.97
answer’s validity	1.54	1.61
answer’s clarity	2.0	2.0
answer’s relevance	1.97	1.98

revise the questions and answers sufficiently to make them valid.

Summary-RQ1: In general, the Generation Agent was able to generate mostly acceptable questions and reference answers, except for the higher cognitive levels of the Bloom’s Taxonomy such as *Evaluate* and *Create*. The Refinement Agent could only manage some marginal improvements mainly in the clarity of the questions and answers. We concluded that it is unnecessary to use the Refinement Agent, as the quality improvement seemed not justifiable given the added token cost and time required for multiple refinement rounds.

2) RQ2: HUMAN RATER VS. LLM BASED RATING AGENTS

We evaluated how the team of three independent Rating Agents implemented in our system compares against the single human rater for all the generated questions/answers. Table 9 shows the numbers of usable questions out of a total of 312 generated questions for all course topics and their corresponding percentages, as determined by the human rater and the team of 3 Rating Agents, respectively. In this work, a question is considered usable if the human rater awards its question text and corresponding reference answers the full rating of 12 points across the six criteria as discussed previously. On average, 79.16% of the generated questions were usable as judged by the human rater. On the other hand, the 3 Rating Agents agreed that 93.91% of all the questions are usable, which was quite optimistic. We also calculated a percentage agreement of 77.56% on the binary decision (usable or not) produced by the single human rater and by the team of 3 Rating Agents (where all agents agreed on the same decision). As the majority of the generated questions was considered usable, a more reliable inter-rater reliability measure like Krippendorff’s Alpha was not necessary in this case.

As observed previously in RQ1, in topics like Automation, Process, and Testing, where most questions focus on explaining or applying established software engineering practices, it is more difficult to develop valid questions targeting higher Bloom’s cognitive levels such as *Evaluate* and *Create*. This explains why there were fewer usable questions in these

TABLE 9. Number and percentage of questions per topic where the human rater and the team of AI raters awarded the full rating of 12 points.

Topic	Human rater		Rating Agents	
	Count	%	Count	%
Total	247	79.16	293	93.91
Automation	47	65.28	66	91.67
Design	34	94.44	34	94.44
Support	17	94.44	18	100.0
Security	34	80.95	42	100.0
Process	22	73.33	28	93.33
XP	33	91.67	35	97.22
Testing	40	74.07	50	92.59
Versioning	20	83.33	20	83.33

TABLE 10. Number of questions per Bloom's categories where the human rater and the team of AI raters awarded the full rating of 12 points.

Bloom's categories	Human rater		Rating Agents	
	Count	%	Count	%
Remember	43	82.69	47	90.38
Understand	48	92.3	49	94.23
Apply	49	94.23	50	96.15
Analyze	45	86.54	50	96.15
Evaluate	38	73.1	51	98.08
Create	24	46.15	46	88.46

topics. In our evaluation, the highly experienced human rater could identify a merely *Apply* question, e.g., designing test cases using boundary values, while all the Rating Agents were probably misled by the keyword used, e.g., “design” and assigned a *Create* category to the same question. On the other hand, topics such as Design could provide more opportunities to create questions asking students to produce novel architectures or software components. Overall, it was clear that the more challenging Bloom's categories had lower percentages of usable questions, as shown in Table 10.

Summary-RQ2: LLM Agents can be reliably used to filter generated questions on lower Bloom's cognitive levels such as *Understand* or *Apply*. Although the overall percentage agreement between the human rater and the team of Rating Agents is more than 77%, the Rating Agents fell short in rating questions designated for higher Bloom's cognitive levels. This suggests that human raters are still needed to filter *Evaluate*- and *Create*-level questions, before students can attempt them. This is important especially if the questions are to be given in high-stake settings such as final exams.

B. ANSWER SCORING

We evaluated the second part of GenScore, which is about auto-scoring short answers. We used a diverse set of datasets including the well-known Mohler dataset, as well as our own datasets, SE dataset and Generated dataset (see Section V-A). The generated dataset was designed for investigating the

TABLE 11. Comparison of our single Scoring Agent with simple embedding and completion based approaches. We used GPT-3.5-turbo model for a fair comparison with existing approaches. The results show that the single Scoring Agent could work well in auto-scoring.

Model	Pearson's	RMSE	MAE
Mohler Dataset			
Embedding	0.557	0.932	0.749
Completion (3 examples)	0.450	1.185	0.960
1 Scoring Agent (3 examples)	0.503	1.118	0.866
Completion (6 examples)	0.406	0.975	0.780
1 Scoring Agent (6 examples)	0.490	1.019	0.781
Completion (9 examples)	0.525	0.922	0.706
1 Scoring Agent (9 examples)	0.528	0.986	0.732
SE Dataset			
Embedding	0.507	2.017	1.727
Completion (3 examples)	0.621	1.342	1.044
1 Scoring Agent (3 examples)	0.706	1.166	0.876
Completion (6 examples)	0.694	1.207	0.872
1 Scoring Agent (6 examples)	0.691	1.107	0.797
Completion (9 examples)	0.674	1.240	0.852
1 Scoring Agent (9 examples)	0.747	0.939	0.610

effect of the Bloom's Taxonomy levels, if any, on the auto-scoring performance.

1) RQ3: A SINGLE SCORING AGENT VS. EXISTING APPROACHES

In this RQ, we investigate if the Scoring Agent implemented using the OpenAI Assistants API could work well for auto-scoring short answers. We compared the performance of the existing embedding and completion based approaches [14] to the new Scoring Agent on both the Mohler dataset [11] and the SE dataset. The embedding based scoring computes cosine similarities between each student answer and all the reference answers using a pre-trained text embedding model. It then derives a predicted score for the student answer which is based on the score of the most similar reference answer. The completion based approach uses the OpenAI Chat Completions API and few-shot prompting. For this approach, integrating relevant context [30] to the scoring process would have to be done manually. Both of these approaches are based on pre-trained LLMs. For a fair comparison to the previous implementation, we used the same LLM model, i.e., GPT-3.5-turbo [31] as the baseline LLM.

As seen in Table 11, for the Mohler dataset [11], most auto-scoring approaches have comparable performances. Notably, the embedding approach achieves the best performance in terms of Pearson's r (higher is better) while the completion approach with 9 examples achieves the best RMSE and MAE results (lower is better). The single Scoring Agent (9 examples) is second best in terms of Pearson's r . On the other hand, for the SE dataset, Table 11 shows that the embedding does not perform as well as other LLM based approaches. This might be due to the fact that the Mohler dataset has more questions and answers about programming concepts which limit the range of possible

TABLE 12. Comparison of LLM based auto-scoring approaches with traditional machine learning based approaches.

Model	Pearson's	RMSE	MAE
Mohler Dataset			
LSTM-EMD-SVOR [32]	0.550	0.830	0.490
LSTM-EMD-Logits [32]	0.649	1.135	0.657
BiLSTM + feature engineering [27]	0.655	0.889	0.618
Stacked BiLSTM (ELMo) [35]	0.485	0.978	-
String + Corpus Similarity [33]	0.470	-	-
Training paragraph vectors [34]	0.569	0.797	-
Embedding	0.557	0.932	0.749
Completion (9 examples)	0.525	0.922	0.706
1 Scoring Agent (9 examples)	0.528	0.986	0.732

answers. This could help the embedding approach, which relies on similarity matching between answers to perform grading. The SE dataset is more about software processes and practices which can have a much wider range of open-ended answers. In this case, the completion approach and the single Scoring Agent can be guided by human-graded examples, which resulted in better grading performance. We note that while more examples could improve grading performance, they could also increase the token cost. In most cases, the single Scoring Agent, which is based on the newer OpenAI Assistants API, had a slight edge over other approaches including the completion approach discussed in [14].

The other notable difference between the two datasets is that the embedding approach's error measures (RMSE and MAE) are roughly twice as high on the SE dataset as on the Mohler dataset [11]. In our experiments, the embedding approach tended to give higher scores due to the relatively high similarity between the student answers and the reference answers in many cases, in particular 86% of the auto-scored answers in the Mohler dataset were given more than 4 marks when using the embedding approach. Looking at the datasets more closely, we noticed that the Mohler dataset's scores are biased towards higher scores with 63% of answers scored above 4 out of 5 marks. On the other hand, the SE dataset is more balanced with only 27% of answers with scores above 3 out of 4 marks. We believed this could also explain why the embedding approach did better for the Mohler dataset than the SE dataset. From this experiment, we noted that the completion and the single Scoring Agent approach could be the better way to do short-answer auto-scoring across datasets with different mark distributions.

As shown in Table 12, we also compared the LLM based approaches to the existing traditional machine learning models for automatic scoring, e.g., the Siamese Bidirectional Long short-term memory (LSTM) [27], [32], String and Corpus-based similarity [33], Training Paragraph vectors model (doc2vec) [34], and pre-trained models such as ELMo [35]. These approaches were evaluated using the Mohler dataset [11]. The LLM based approaches all used GPT-3.5-turbo model.

Although the performance of the LLM based approaches might not be the best when compared to the existing models,

TABLE 13. Comparison: passing additional context in the form of relevant chunks of lecture notes to the auto-scoring approaches using GPT-3.5-turbo. The results show that the single Scoring Agent could work better with relevant context.

Approach	Pearson's	RMSE	MAE
SE Dataset			
Completion (3 examples)	0.621	1.342	1.044
Completion with Context (3 examples)	0.631	1.338	1.018
Completion (6 examples)	0.694	1.207	0.872
Completion with Context (6 examples)	0.642	1.149	0.795
Completion (9 examples)	0.674	1.240	0.852
Completion with Context (9 examples)	0.748	1.026	0.693
1 Scoring Agent (3 examples)	0.706	1.166	0.876
1 Scoring Agent with Context (3 examples)	0.725	1.156	0.909
1 Scoring Agent (6 examples)	0.691	1.107	0.797
1 Scoring Agent with Context (6 examples)	0.693	1.176	0.881
1 Scoring Agent (9 examples)	0.747	0.939	0.610
1 Scoring Agent with Context (9 examples)	0.760	1.020	0.705

all three approaches provided a great balance between the three scoring metrics and their performance is average among all the approaches. We should note that the LLM based approaches require no extensive training or fine-tuning with a large labeled dataset. On the other hand, machine learning based approaches incurred significant training cost prior to predictions [36]. In some cases, e.g., [27], extensive manual feature engineering was required to improve the prediction performance.

Next, we wanted to find out if there would be an improvement in the auto-scoring capabilities when relevant context extracted from course materials are provided to the auto-scoring process. This is referred to as retrieval augmented generation (RAG) [30]. The results are shown in Table 13. The OpenAI Assistants API allows for tool usage, in particular built-in file search to automatically retrieve context from external data sources, which is more convenient compared to custom-built RAG pipelines as implemented previously in [14]. For comparison purpose, we also included the results from [14] in Table 13, which used OpenAI's *text-embedding-ada-002* and the Faiss library [37] to store and extract chunks of lecture notes relevant to a question for which its answers need to be auto-scored. The chunks were then incorporated into the prompt together with the grading examples.

We observed notable improvements in the quality of auto-scoring for short answers when using additional relevant context from lecture notes in our course. For instance, the completion approach with 9 examples (no context provided) produced a Pearson's r of 0.674, RMSE of 1.207 and MAE of 0.872. With relevant context, the same approach did better with a Pearson's r of 0.748, RMSE of 1.026 and MAE of 0.693. Similarly, additional context improved auto-scoring performance for the single Scoring Agent. In particular, the best performing approach in terms of Pearson's r is now the single Scoring Agent (with 9 examples), as shown in Table 13. We also noted that using a better (at the time of the experiment) GPT model such as GPT-4o-mini together with relevant context could improve the scoring performance significantly, as shown in Table 14.

TABLE 14. Performance for the single Scoring Agent with Context, using GPT-3.5-turbo and GPT-4o-mini. The newer and more expensive model at the time of experiment did perform better.

Model	Pearson's	RMSE	MAE
SE Dataset			
GPT-3.5-turbo (3 examples)	0.725	1.156	0.909
GPT-4o-mini (3 examples)	0.759	1.046	0.767
GPT-3.5-turbo (6 examples)	0.693	1.176	0.881
GPT-4o-mini (6 examples)	0.745	0.946	0.664
GPT-3.5-turbo (9 examples)	0.760	1.020	0.705
GPT-4o-mini (9 examples)	0.783	0.910	0.659

TABLE 15. Comparison of performance across different Bloom's categories using the single Scoring Agent approach with GPT-4o-mini, based on the Generated dataset.

Bloom's Level	Pearson's	RMSE	MAE
1 Scoring Agent (3 examples)			
Create	0.857	0.607	0.474
Analyze	0.882	0.631	0.446
Remember	0.869	0.644	0.425
1 Scoring Agent (6 examples)			
Create	0.879	0.649	0.531
Analyze	0.919	0.590	0.466
Remember	0.878	0.555	0.331
1 Scoring Agent (9 examples)			
Create	0.812	0.680	0.558
Analyze	0.920	0.506	0.397
Remember	0.919	0.437	0.281

As part of this RQ, we also investigated how questions under different Bloom's categories might affect the auto-scoring performance of the single Scoring Agent. We used the Generated dataset (Section V-A), in which questions were auto-generated by the Generation Agent as described earlier. Compared to the SE dataset, which mostly consists of questions under the lower cognitive levels of the Bloom taxonomy, the Generated dataset consists of questions under higher Bloom's cognitive levels. More specifically, 2 questions in the Generated dataset are under *Remember*, 2 are under *Create*, and 2 are under *Analyze* categories. The auto-scoring performance across Bloom's levels for the Generated dataset is given in Table 15. We note that the Bloom's levels did not seem to have a significant effect on the auto-scoring performance.

Summary-RQ3: Generally, the LLM based single Scoring Agent was better in our experiments as it performed more consistently across datasets with varying mark distributions. LLM based approaches did not require extensive training and fine-tuning to perform reasonably well. Providing examples of human assessment and relevant context from lecture notes could significantly improve the auto-scoring performance. On the other hand, the Bloom's categories did not seem to have a significant effect on the auto-scoring performance.

TABLE 16. Comparison of a single Scoring Agent and multiple-agent approaches. All agents used relevant context extraction with GPT-3.5-turbo. Multiple agents provided modest improvement with more cost (3x or 6x more depending on the number of agents).

Model	Pearson's	RMSE	MAE
SE Dataset			
1 Scoring Agent (3 examples)	0.725	1.156	0.909
3 Scoring Agents (3 examples)	0.678	1.223	0.948
6 Scoring Agents (3 examples)	0.720	1.154	0.932
1 Scoring Agent (6 examples)	0.693	1.176	0.881
3 Scoring Agents (6 examples)	0.689	1.147	0.762
6 Scoring Agents (6 examples)	0.728	1.003	0.762
1 Scoring Agent (9 examples)	0.760	1.020	0.705
3 Scoring Agents (9 examples)	0.765	1.029	0.717
6 Scoring Agents (9 examples)	0.758	0.983	0.752

Finally, better LLMs, e.g., GPT-4o-mini, generally provide better grading performance.

2) RQ4: USING MULTIPLE SCORERS

In this RQ, we evaluated if using multiple Scoring Agents would improve performance. We used 3 and 6 identical and independent Scoring Agents to do scoring on the same datasets, and reported the average scores. All agents were provided with relevant context via the Assistants API. The results are shown in Table 16.

Based on the Pearson's r , the best approach would be the 3 Scoring Agents (9 examples, $r = 0.765$) while the RMSE and MAE scores suggest that the 6 Scoring Agents (9 examples, $RMSE = 0.983$), and the single Scoring Agent (9 examples, $MAE = 0.705$) were the best, respectively. However, we noted that the improvement of the multi-agent approach was quite modest compared to the single-agent approach. On the other hand, the cost of multi-agent was much more (3x or 6x more depending on how many agents were employed). Overall, the 3 Scoring Agents (9 examples) seem to strike a good balance between all the three performance measures (Pearson's r , RMSE, and MAE).

We further investigated the performance of the 3 Scoring Agents when there was no relevant course materials to be used as context. For this purpose, we used the Mohler dataset. The results are shown in Table 17. For comparison purposes, this table also includes the results from the best deep learning and LLM based approaches for scoring the Mohler dataset in the last three rows. We observed that the 3 Scoring Agents could significantly improve the performance compared to the single agent approach (with 9 examples, no context). Furthermore, the 3 Scoring Agents with an older LLM like GPT-3.5-turbo actually outperformed (in terms of RMSE and MAE) the existing deep learning based approaches, e.g., [27] which require extensive feature engineering and training.

To investigate the robustness of the auto-scoring process when using multiple agents, we computed the inter-rater reliability (IRR) between the 3 Scoring Agents. The IRR values were computed using percentage agreement and the more rigorous Krippendorff's Alpha measure. The results

TABLE 17. Comparison of the 3 Scoring Agents (using GPT-3.5-turbo and GPT-4o-mini) against the existing deep learning based approaches, and the single Scoring Agent. No context extraction was used in this experiment. The 3 Scoring Agents did not require context to perform well for the Mohler dataset.

Model	Pearson's	RMSE	MAE
3 Scoring Agents			
GPT-3.5-turbo (3 examples)	0.571	0.987	0.770
GPT-4o-mini (3 examples)	0.596	0.906	0.673
GPT-3.5-turbo (6 examples)	0.576	0.845	0.626
GPT-4o-mini (6 examples)	0.600	0.787	0.558
GPT-3.5-turbo (9 examples)	0.631	0.751	0.544
GPT-4o-mini (9 examples)	0.628	0.744	0.512
Deep learning approaches			
LSTM-EMD-Logits [32]	0.649	1.135	0.657
BiLSTM + feature engineering [27]	0.655	0.889	0.618
1 Scoring Agent			
1 Scoring Agent (9 examples)	0.528	0.986	0.732

TABLE 18. Inter-rater reliability between 3 Scoring Agents. The newer GPT-4o-mini had much better IRR, which means it was more consistent in grading compared to the older LLM.

Model	% Agreement	Krippendorff's Alpha
Mohler Dataset		
GPT-3.5-turbo (3 examples)	0.536	0.770
GPT-4o-mini (3 examples)	0.788	0.945
GPT-3.5-turbo (6 examples)	0.549	0.784
GPT-4o-mini (6 examples)	0.810	0.957
GPT-3.5-turbo (9 examples)	0.565	0.808
GPT-4o-mini (9 examples)	0.787	0.946
SE Dataset		
GPT-3.5-turbo (3 examples)	0.512	0.608
GPT-4o-mini (3 examples)	0.797	0.904
GPT-3.5-turbo (6 examples)	0.572	0.658
GPT-4o-mini (6 examples)	0.798	0.886
GPT-3.5-turbo (9 examples)	0.563	0.723
GPT-4o-mini (9 examples)	0.779	0.903

are shown in Table 18. In many cases, IRR values showed improvement as the number of examples increases. It means that using more examples enables the agents to score more consistently. We also note that the newer LLM (GPT-4o-mini) allows the agents to score in a more consistent manner compared to the older model (GPT-3.5-turbo).

Summary-RQ4: Provided with relevant context and examples, a single Scoring Agent is capable of auto-scoring short answers in our software engineering course when compared to multiple Scoring Agents. Without course materials for context, using multiple agents and averaging their outputs could enhance the auto-scoring performance. From the experiments, we recommend using three independent Scoring Agents, which strikes a good balance between scoring accuracy and cost. Finally, when using multiple agents, it would be better to employ newer LLM versions for better scoring consistency.

C. COST ANALYSIS

This section provides insights into the feasibility in terms of costs for a large-scale deployment of our proposed GenScore system. Using external LLMs like those offered by OpenAI involves API costs which are calculated on a

TABLE 19. Average token counts and costs per generated question for the question generation agents using GPT-4o-mini.

Agent	Input Tokens	Output Tokens	Total Cost (\$)
Generation	18148	884	0.0032
Refinement (3 rounds)	54981	1456	0.0091
Rating (3 agents)	54354	536	0.0084

TABLE 20. Average token counts and costs per scored answer across datasets and scoring approaches (single Scoring Agent).

Model	Input Tokens	Output Tokens	Total Cost (\$)
Agile/DevOps Dataset			
GPT-3.5-turbo-0125 (3 examples)	1070	10	0.00082
GPT-4o-mini (3 examples)	1397	10	0.00049
GPT-3.5-turbo-0125 (6 examples)	1359	10	0.00097
GPT-4o-mini (6 examples)	1366	10	0.00048
GPT-3.5-turbo-0125 (9 examples)	1633	10	0.0011
GPT-4o-mini (9 examples)	1642	10	0.00052
Agile/DevOps-2 Dataset			
GPT-4o-mini (3 examples)	1348	10	0.00047
GPT-4o-mini (6 examples)	1872	10	0.00055
GPT-4o-mini (9 examples)	2355	10	0.00062
Generated Dataset			
GPT-4o-mini (3 examples)	1228	10	0.00045
GPT-4o-mini (6 examples)	1645	10	0.00051
GPT-4o-mini (9 examples)	2010	10	0.00057

TABLE 21. Average token counts and costs per scored answer for the Mohler dataset using 3 Scoring Agents. The results for other datasets are similar, i.e., a 3x increase in cost.

Model	Input Tokens	Output Tokens	Total Cost (\$)
Mohler Dataset			
GPT-3.5 (3 examples)	2994	30	0.0015
GPT-4o-mini (3 examples)	3035	30	0.00047
GPT-3.5 (6 examples)	3753	30	0.0019
GPT-4o-mini (6 examples)	3809	30	0.00059
GPT-3.5 (9 examples)	4533	30	0.0023
GPT-4o-mini (9 examples)	4584	30	0.00071

per-token basis.¹ OpenAI charges differently for input and output tokens; the latter are more expensive as described in Section V-C. Tables 19, 20, and 21 summarize the average number of tokens used and the corresponding costs (in USD) in our experiments.

We observed that in both question generation and scoring, the input tokens dominated the cost. This is because a large amount of context tokens from relevant lecture notes are required to be given as input to the LLM prompts for generating new questions and scoring answers. On the other hand, the outputs of the agents include short questions and sample answers, refined questions, rating, and scores. Such outputs do not use as many tokens.

In the question generation experiments, around 5.6 millions of input tokens were used by the Generation Agent to generate a total of 312 questions. The Refinement Agent (with 3 rounds of refinement) and the team of Rating Agents (3 of them) each used about 3x this number of input tokens. On the other hand, the total number of output tokens produced by the Generation Agent, Refinement Agent, and Rating Agents was much less compared to the number of input tokens.

¹<https://platform.openai.com/docs/pricing>

For the auto-scoring experiments, we noted that much less input tokens were used on average for the single Scoring Agent, as only context relevant to the question was extracted from the course materials to score an answer. This is shown in Table 20. There were about 10 output tokens for each scoring task, which is negligible in terms of cost. For multiple Scoring Agents, as shown in Table 21, the cost would increase proportionally, e.g., using 3 independent Scoring Agents results in around 3x increase in the token cost (both input and output).

Given that 1 million of input tokens for GPT-4o-mini cost \$0.15, we can see from the tables that the average cost per generated question, or per scored answer is rather small. This would make the cost of our system more manageable when we need to do a real-world deployment to hundreds of students in a software engineering course. We also noted that at the time of writing, the cost of the older model GPT-3.5-turbo is about 3x the cost of the better and smaller GPT-4o-mini model. Therefore, we recommend to use GPT-4o-mini as the default LLM for our system deployment until stronger and cheaper LLMs are evaluated for the specific use cases considered in this work.

D. THREATS TO VALIDITY

Like other empirical studies, the generalizability of our results is subject to a few threats as discussed below.

First, this study employed a single human rater for the manual evaluation of generated questions. The rater is highly experienced in software engineering educational assessments. Such expertise, especially in highly specialized domains like software engineering education, can provide reliable evaluation of outputs generated by the agents in GenScore. However, we also recognized that including a second rater (who was not available to us during the time of the experiments) and reporting inter-rater metrics such as Krippendorff's alpha might strengthen the reliability of this study.

Second, outputs from LLMs could be changing from time to time due to their probabilistic nature. This would affect the quality of question generation, the rating of questions and the auto-scoring accuracy. We have mitigated this issue by generating and evaluating more than 300 new questions across 8 popular topics in software engineering and 6 different Bloom's cognitive levels. We also used three different datasets with a total of more than 2800 answers to be auto-scored. At the same time, we recognize the importance of conducting the evaluation with more real-world datasets containing questions set at the higher cognitive levels of the Bloom categories, especially for the auto-scoring performance. Such datasets in software engineering courses are not available to us at the moment.

Third, due to the budget and time constraints, we could not fully implement and evaluate the latest LLM models such as the reasoning models [38]. Newer and more expensive LLM models might improve the generation and auto-scoring performance compared to what have been reported in this

work. As an extension of this work, we plan to run experiments with the latest LLM models such as OpenAI's GPT-5 when the cost becomes more manageable and study if it improves the performance of rating agents when dealing with the higher cognitive levels of Bloom categories. In addition, we also plan to build rubric-based generators or discriminator enhancements designed specifically for higher Bloom levels such as *Evaluate* and *Create*, to improve our current rating agents. Such enhancements may produce better agreement between the human rater and the LLM-based agents. We note this area as an opportunity for possible improvement in our upcoming studies.

Finally, we acknowledge that GenScore is primarily a proof of concept system. We plan to make it production-ready and evaluate it further in terms of system scalability and usability in real classroom deployments. In order not to cause possibly significant disruptions in student learning, such deployments will be complementary to existing classroom practices, i.e., students will have GenScore as an additional tool to practice new short answer questions in their own pace and get instant feedback. Based on the experimental results, we believe that it is necessary to include expert human oversight over practice questions generated by the system under the higher Bloom cognitive levels.

VII. RELATED WORK

Auto-generating assessments is an established research topic in technology-enabled education [39]. Existing work in this area have been focusing on various educational domains, such as reading comprehension question generation using natural language processing techniques [40], and more recently LLM based programming exercise generation [41], [42]. In [8], the authors applied LLMs and knowledge graphs to generate questions and answers for cybersecurity training. In [9], the authors performed automatic generation and evaluation of questions for content in Spanish. We have not come across works on auto-generating short-answer questions for software engineering courses.

To the best of our knowledge, not much work has been done in generating short-answers questions that are aligned to the Bloom's Taxonomy, which has been considered widely in educational assessment [43]. In [44], the authors investigated the problem of generating questions for the 6-level Bloom's Taxonomy based on the lecture material of an undergraduate software engineering course. The study evaluated 120 generated questions, but they were essay-type, not short-answer type of questions. In a more recent study [10], the authors used GPT-4 for Bloom-aligned multiple-choice, not short-answer, question generation in a biology course.

There have been recent research on the effectiveness of LLMs in generating assessment questions based on taxonomies such as Bloom's. In [45], the authors conducted a human evaluation to assess the usefulness and quality of the questions generated by LLMs. The questions used in the evaluation were in the machine learning and biology domains, not software engineering. They showed that the

questions generated were of good quality. In GenScore, we implemented a multi-agent based approach to rate the generated questions and compared it to human-based rating. Our evaluation showed that it might be harder for LLMs to generate high quality questions at the higher cognitive levels such as *Evaluate* or *Create* in Bloom's Taxonomy, especially in the more theory-oriented parts of software engineering courses.

On the other hand, auto-scoring of short textual answers is a relatively more well-studied problem, especially with the recent development in LLMs [15]. Previous approaches made use of machine learning techniques [11], [12], [13], in which significant effort would be needed to manually engineer appropriate features for training the machine learning models. In [12], using the Mohler dataset [11] the authors considered different feature extraction techniques including text similarity, weights of terms, question demotion, etc., to develop regression models for auto-scoring short answers. In [13], various traditional machine learning models such as Random Forest and Support Vector Machines were trained on a subset of human-graded answers to classify the remaining answers. The results demonstrated significant reduction in manual workload in grading. Interestingly, in a recent work (2025), Boonsarngsuk et al. [46] found that simple reference answer sampling strategies such as K-means clustering and K-center outperformed existing similarity-based auto-scoring approaches. They achieved competitive performance compared to new LLM based approaches using OpenAI's GPT-4o-mini. Their datasets were collected from a Statistics and a Computer Architecture course at a university in Thailand.

To reduce the manual feature engineering effort required in traditional machine learning, deep learning based methods for auto-scoring have become popular [32], [34], [36], [47]. Such models, e.g., text embedding, attention based neural networks, etc., have the ability to learn representative features directly from large datasets. In addition, manually crafted features could be incorporated in a complementary manner into deep learning models to enhance scoring accuracy [27]. We noted that deep learning based approaches require large, high quality datasets for proper model training and evaluation.

With the recent rise of LLMs, naturally there have been more work on using them to perform auto-scoring of open-ended answers, for instance [14], [15], [16], [17]. Ta et al. [14] evaluated the performance of different OpenAI's LLMs in scoring short-answer questions in software engineering courses, but did not consider an agent based approach nor the integration with question generation in a unified system to support learning. Other than that, not much work has been done for LLM-based auto-scoring of short answers in the context of computing education, especially in more theory-oriented software engineering courses. Research in auto-scoring of open-ended answers in computing education have been more involved with programming based questions [19], [48].

In other educational domains, Chang and Ginter [15] investigated the auto-scoring problem in Finnish based undergraduate courses using GPT-3.5 and GPT-4. The data was collected from different academic disciplines at the University of Turku. They found that GPT-4 (one-shot learning) provided the best performance. In [49], the authors implemented and evaluated few-shot pre-trained language models for the automatic essay scoring task. We noted that grading essays is quite different from short-answer scoring, as the former requires the evaluation of different criteria such as text coherence and organization, logical arguments, etc. The authors of [50] evaluated GPT-4 for short answer grading using the SciEntsBank and Beetle datasets. They found that for these datasets, GPT-4's performance is comparable to manually crafted machine learning models. ChatGPT was used in [51] to correct open-ended answers in technical training of professional software developers. We noted that existing approaches did not attempt to investigate the auto-scoring performance with respect to various cognitive levels in the Bloom's Taxonomy.

Summary: Our approach presents several main novelties over existing LLM-based educational tools. First, GenScore integrates question generation and auto-scoring in a unified system for software engineering education, whereas existing work treats these as separate problems. Second, we target short-answer questions aligned to Bloom's Taxonomy for theory-oriented software engineering courses. Prior work has focused on other assessment formats (essays, multiple-choice, programming exercises) or other educational domains. Third, we evaluate both question generation quality and scoring performance across Bloom's cognitive levels. Our experiments revealed that LLMs struggle more with creating questions for higher-order thinking skills such as *Evaluate* and *Create* in software engineering contexts. This taxonomy-based performance analysis for both generation and scoring has not been explored in prior research in automated assessments.

VIII. CONCLUSION

In this work, we developed a practical, automated short answer question generation and answer scoring system, named GenScore, which can be used by both instructors and students in the context of software engineering courses. GenScore involves the use of LLM agents which can perform complex tasks based on customized prompts and relevant tool usage, e.g., to extract context from suitable lecture notes for enhancing generation and scoring performance. In this way, GenScore can be used for question generation and scoring for a wide variety of courses if the relevant course materials are provided, not just for software engineering courses.

We conducted extensive experiments to investigate the auto-generation and auto-scoring capabilities of GenScore. We used a large number of course topics and questions/answers from various datasets including publicly available and our own datasets. The results demonstrated

the viability of generating new short-answer questions across the cognitive levels of the widely used Bloom's Taxonomy in education. We also showed that LLM agents, when implemented properly with relevant context and a multi-agent approach, could perform well in auto-scoring even with older LLMs such as GPT-3.5-turbo. GenScore can easily incorporate new AI techniques and models, which are being introduced on a weekly basis these days, to further reduce workload for instructors in tedious question creation and grading tasks.

ACKNOWLEDGMENT

The authors would like to thank Joel Ow and Eugene Lian for their initial contributions to this work. We would also like to thank Li Ka Shing Library, Singapore Management University for supporting the publication of this work.

GenScore generated the questions and model answers in Listings 6, 7, and 8 based on OpenAI's GPT-4o-mini [52].

REFERENCES

- [1] T. Puthiamparmpil and M. M. Rahman, "Very short answer questions: A viable alternative to multiple choice questions," *BMC Med. Educ.*, vol. 20, no. 1, pp. 1–8, Dec. 2020.
- [2] S. Greiving and T. Richter, "Examining the testing effect in university teaching: Retrieval practice and question format matter," *Frontiers Psychol.*, vol. 9, Dec. 2018. [Online]. Available: <https://www.frontiersin.org/journals/psychology/articles/10.3389/fpsyg.2018.02412/full>
- [3] H. L. Roediger, P. K. Agarwal, M. A. McDaniel, and K. B. McDermott, "Test-enhanced learning in the classroom: Long-term improvements from quizzing," *J. Experim. Psychol., Appl.*, vol. 17, no. 4, pp. 382–395, 2011.
- [4] P. Armstrong, "Bloom's taxonomy," Vanderbilt Univ. Center for Teaching, Tech. Rep., 2010. [Online]. Available: <https://ies.ed.gov/ncee/rel/regions/northeast/onlinelearning/ResourcesTools/Bloom's%20Taxonomy.pdf>
- [5] P. K. Agarwal, "Retrieval practice & Blooms taxonomy: Do students need fact knowledge before higher order learning?" *J. Educ. Psychol.*, vol. 111, no. 2, p. 189, 2019.
- [6] G.-G. Lee, E. Latif, X. Wu, N. Liu, and X. Zhai, "Applying large language models and chain-of-thought for automatic scoring," *Comput. Educ., Artif. Intell.*, vol. 6, Jun. 2024, Art. no. 100213.
- [7] C. Stamford. (Jul. 2024). *Gartner Forecasts Worldwide It Spending To Grow 7.5% in 2024*. [Online]. Available: <https://tinyurl.com/e7v4x6x2>
- [8] G. Agrawal, K. K. Pal, Y. Deng, H. Liu, and Y. Chen, "CyberQ: Generating questions and answers for cybersecurity education using knowledge graph-augmented LLMs," in *Proc. AAAI Conf. Artif. Intell.*, 2024, vol. 38, no. 21, pp. 23164–23172.
- [9] B. G. Johnson, R. Van Campenhout, B. Jerome, M. F. Castro, R. Bistolfi, and J. S. Dittel, "Automatic question generation for Spanish textbooks: Evaluating Spanish questions generated with the parallel construction method," *Int. J. Artif. Intell. Educ.*, vol. 35, no. 3, pp. 1–20, Sep. 2025.
- [10] K. Hwang, K. Wang, M. Alomair, F.-S. Choa, and L. K. Chen, "Towards automated multiple choice question generation and evaluation: Aligning with Blooms taxonomy," in *Proc. Int. Conf. Artif. Intell. Educ.*, 2024, pp. 389–396.
- [11] M. Mohler, R. Bunesco, and R. Mihalcea, "Learning to grade short answer questions using semantic similarity measures and dependency graph alignments," in *Proc. 49th Annu. Meeting Assoc. Comput. Linguistics, Human Lang. Technol.*, 2011, pp. 752–762.
- [12] M. A. Sultan, C. Salazar, and T. Sumner, "Fast and easy short answer grading with high accuracy," in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics, Human Lang. Technol.*, 2016, pp. 1070–1075.
- [13] R. Weegar and P. Idestam-Almqvist, "Reducing workload in short answer grading using machine learning," *Int. J. Artif. Intell. Educ.*, vol. 34, no. 2, pp. 247–273, Jun. 2024.
- [14] T. N. B. Duong and C. Y. Meng, "Automatic grading of short answers using large language models in software engineering courses," in *Proc. IEEE Global Eng. Educ. Conf. (EDUCON)*, May 2024, pp. 1–10.
- [15] L.-H. Chang and F. Ginter, "Automatic short answer grading for Finnish with ChatGPT," in *Proc. AAAI Conf. Artif. Intell.*, 2024, vol. 38, no. 21, pp. 23173–23181.
- [16] C. Grévisse, "LLM-based automatic short answer grading in undergraduate medical education," *BMC Med. Educ.*, vol. 24, no. 1, p. 1060, Sep. 2024.
- [17] K. Cochran, C. Cohn, J. F. Rouet, and P. Hastings, "Improving automated evaluation of student text responses using gpt-3.5 for text data augmentation," in *Proc. Int. Conf. Artif. Intell. Educ.*, 2023, pp. 217–228.
- [18] S. T. Himi, N. Tanzila Monalisa, S. Sultana, A. Afrin, and K. M. Hasib, "Automated exam script checking using zero-shot LLM and adaptive generative AI," in *Proc. IEEE Int. Conf. Comput., Appl. Syst. (COMPAS)*, Sep. 2024, pp. 1–6.
- [19] D. S. Mishra and S. H. Edwards, "The programming exercise markup language: Towards reducing the effort needed to use automated grading tools," in *Proc. 54th ACM Tech. Symp. Comput. Sci. Educ. V. 1*, Mar. 2023, pp. 395–401.
- [20] E. Thompson, A. Luxton-Reilly, J. L. Whalley, M. Hu, and P. Robbins, "Bloom's taxonomy for cs assessment," in *Proc. 10th Conf. Australas. Comput. Educ.*, 2008, pp. 155–161.
- [21] R. Wang, J. Guo, C. Gao, G. Fan, C. Y. Chong, and X. Xia, "Can LLMs replace human evaluators? An empirical study of LLM-as-a-judge in software engineering," *Proc. ACM Softw. Eng.*, vol. 2, no. ISSTA, pp. 1955–1977, Jun. 2025.
- [22] J. Qiu, K. Lam, G. Li, A. Acharya, T. Y. Wong, A. Darzi, W. Yuan, and E. J. Topol, "LLM-based agentic systems in medicine and healthcare," *Nature Mach. Intell.*, vol. 6, no. 12, pp. 1418–1420, Dec. 2024.
- [23] J. He, C. Treude, and D. Lo, "LLM-based multi-agent systems for software engineering: Literature review, vision, and the road ahead," *ACM Trans. Softw. Eng. Methodol.*, vol. 34, no. 5, pp. 1–30, Jun. 2025.
- [24] C. Gao, X. Lan, N. Li, Y. Yuan, J. Ding, Z. Zhou, F. Xu, and Y. Li, "Large language models empowered agent-based modeling and simulation: A survey and perspectives," *Humanities Social Sci. Commun.*, vol. 11, no. 1, pp. 1–24, 2024.
- [25] B. Yao, G. Chen, R. Zou, Y. Lu, J. Li, S. Zhang, Y. Sang, S. Liu, J. Hendler, and D. Wang, "More samples or more prompts? Exploring effective few-shot in-context learning for LLMs with in-context sampling," in *Proc. Findings Assoc. Comput. Linguistics, NAACL*, 2024, pp. 1772–1790.
- [26] I. Moreno-Medina, M. Peñas-Garazón, C. Belver, and J. Bedia, "Wooclap for improving student achievement and motivation in the chemical engineering degree," *Educ. Chem. Eng.*, vol. 45, pp. 11–18, Oct. 2023.
- [27] A. Prabhudesai and T. N. B. Duong, "Automatic short answer grading using Siamese bidirectional LSTM based regression," in *Proc. IEEE Int. Conf. Eng., Technol. Educ. (TALE)*, Dec. 2019, pp. 1–6.
- [28] J. Belur, L. Tompson, A. Thornton, and M. Simon, "Interrater reliability in systematic review methodology: Exploring variation in coder decision-making," *Sociol. Methods Res.*, vol. 50, no. 2, pp. 837–865, May 2021.
- [29] K. Krippendorff, *The Reliability of Generating Data*. London, U.K.: Chapman & Hall, 2022.
- [30] J. Jin, Y. Zhu, Z. Dou, G. Dong, X. Yang, C. Zhang, T. Zhao, Z. Yang, and J.-R. Wen, "FlashRAG: A modular toolkit for efficient retrieval-augmented generation research," in *Proc. ACM Web Conf.*, May 2025, pp. 737–740.
- [31] (2024). *OpenAI GPT-3.5-Turbo*. [Online]. Available: <https://platform.openai.com/docs/models>
- [32] S. Kumar, S. Chakrabarti, and S. Roy, "Earth mover's distance pooling over Siamese LSTMs for automatic short answer grading," in *Proc. 26th Int. Joint Conf. Artif. Intell.*, Aug. 2017, pp. 2046–2052.
- [33] W. H. Gomaa and A. A. Fahmy, "Short answer grading using string similarity and corpus-based similarity," *Int. J. Adv. Comput. Sci. Appl.*, vol. 3, no. 11, pp. 115–121, 2012, doi: [10.14569/ijacsa.2012.031119](https://doi.org/10.14569/ijacsa.2012.031119). [Online]. Available: https://thesai.org/Downloads/Volume3No11/Paper_19-Short_Answer_Grading_Using_String_Similarity_And_Corpus-Based_Similarity.pdf
- [34] S. Hassan, A. A. Fahmy, and M. El-Ramly, "Automatic short answer scoring based on paragraph embeddings," *Int. J. Adv. Comput. Sci. Appl.*, vol. 9, no. 10, pp. 397–402, 2018. [Online]. Available: https://thesai.org/Downloads/Volume9No10/Paper_48-Automatic_Short_Answer_Scoring.pdf
- [35] S. Kiran Gaddipati, D. Nair, and P. G. Plöger, "Comparative evaluation of pretrained transfer learning models on automatic short answer grading," 2020, *arXiv:2009.01303*.

- [36] X. Zhu, H. Wu, and L. Zhang, "Automatic short-answer grading via BERT-based deep neural networks," *IEEE Trans. Learn. Technol.*, vol. 15, no. 3, pp. 364–375, Jun. 2022.
- [37] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," *IEEE Trans. Big Data*, vol. 7, no. 3, pp. 535–547, Jul. 2021.
- [38] K. Kumar, T. Ashraf, O. Thawakar, R. Muhammad Anwer, H. Cholakal, M. Shah, M.-H. Yang, P. H. S. Torr, F. Shahbaz Khan, and S. Khan, "LLM post-training: A deep dive into reasoning large language models," 2025, *arXiv:2502.21321*.
- [39] B. Das, M. Majumder, S. Phadikar, and A. A. Sekh, "Automatic question generation and answer assessment: A survey," *Res. Pract. Technol. Enhanced Learn.*, vol. 16, no. 1, p. 5, Dec. 2021.
- [40] Y. Huang and L. He, "Automatic generation of short answer questions for reading comprehension assessment," *Natural Lang. Eng.*, vol. 22, no. 3, pp. 457–489, May 2016.
- [41] A. Del Carpio Gutierrez, P. Denny, and A. Luxton-Reilly, "Evaluating automatically generated contextualised programming exercises," in *Proc. 55th ACM Tech. Symp. Comput. Sci. Educ. V. 1*, Mar. 2024, pp. 289–295.
- [42] N. B. D. Ta, H. L. Nguyen, and S. Gottipati, "ExGen: Ready-to-use exercise generation in introductory programming courses," in *Proc. Int. Conf. Comput. Educ.*, 2023, pp. 104–113. [Online]. Available: <https://library.apsce.net/index.php/ICCE/article/view/953/4678>
- [43] A. Momen, M. Ebrahimi, and A. M. Hassan, "Importance and implications of theory of Bloom's taxonomy in different fields of education," in *Proc. Int. Conf. Emerg. Technol. Intell. Syst.*, 2022, pp. 515–525.
- [44] B. D. Wijanarko, Y. Heryadi, H. Toba, and W. Budiharto, "Question generation model based on key-phrase, context-free grammar, and Bloom's taxonomy," *Educ. Inf. Technol.*, vol. 26, no. 2, pp. 2207–2223, Mar. 2021.
- [45] S. Elkins, E. Kochmar, J. C. K. Cheung, and I. V. Serban, "How useful are educational questions generated by large language models?" in *Proc. Int. Conf. Artif. Intell. Educ.*, 2023, pp. 536–542.
- [46] P. Boonsarnsuk, P. Arpanantikul, S. Hiranwipap, W. Watcharakajorn, and E. Chuangsuanich, "Evaluating sampling strategies for similarity-based short answer scoring: A case study in Thailand," in *Proc. 2nd Workshop South East Asian Lang. Process.*, 2025, pp. 27–41.
- [47] S. Haller, A. Aldea, C. Seifert, and N. Strisciuglio, "Survey on automated short answer grading with deep learning: From word embeddings to transformers," 2022, *arXiv:2204.03503*.
- [48] C. Koutchene, N. Dainese, S. Sarsa, A. Hellas, J. Leinonen, S. Ashraf, and P. Denny, "Evaluating language models for generating and judging programming feedback," in *Proc. 56th ACM Tech. Symp. Comput. Sci. Educ. V. 1*, Feb. 2025, pp. 624–630.
- [49] T. Amin, Z.-U.-R. Tanoli, F. Aadil, K. M. Awan, and S. Lim, "Enhancing essay scoring: An analytical and holistic approach with few-shot transformer-based models," *IEEE Access*, vol. 13, pp. 12483–12501, 2025.
- [50] G. Kortemeyer, "Performance of the pre-trained large language model GPT-4 on automated short answer grading," *Discover Artif. Intell.*, vol. 4, no. 1, p. 47, Jul. 2024.
- [51] G. Pinto, I. Cardoso-Pereira, D. Monteiro, D. Lucena, A. Souza, and K. Gama, "Large language models for education: Grading open-ended questions using ChatGPT," in *Proc. 37th Brazilian Symp. Softw. Eng.*, Sep. 2023, pp. 293–302.
- [52] (2024). *OpenAI GPT-4o-MINI*. [Online]. Available: <https://platform.openai.com/docs/models/gpt-4o-mini>



TA NGUYEN BINH DUONG received the Ph.D. degree in computer science from Nanyang Technological University (NTU), Singapore. He is currently an Associate Professor in computer science (education) with the School of Computing and Information Systems, Singapore Management University. His main research interests include AI-enabled education and distributed computing. He was a recipient of the NTU Tan Chin Tuan Exchange Fellowship, in 2017, the Best Paper Award Nomination from the 10th IEEE Service Oriented Computing and Applications, in 2017, and the Best Technical Design Paper Award from the 31st International Conference on Computers in Education, in 2023.



LWIN KHIN SHAR (Member, IEEE) received the Ph.D. degree from Nanyang Technological University (NTU), Singapore. He was a Postdoctoral Research Associate with SnT, University of Luxembourg, and then a Research Scientist with NTU. He is currently an Associate Professor in computer science (practice) with the School of Computing and Information Systems, Singapore Management University. Particularly, he focuses on analyzing and detecting security vulnerabilities, privacy issues, malware, and anomalies. His research interests include security testing and analysis of web applications, mobile applications, and recently cyber-physical systems. In his research, he often collaborates with industry partners spanning from healthcare to government sectors. He is a member of ACM. His recent awards include the runner-up of the IEEE Best Paper Awards, in 2021, and the Best Paper Award nomination in ITiCSE, in 2022.

...